`brain/carl_reflex.py` — Full Line-by-Line Walkthrough

Length: 177 lines. **Role:** fast, hard-wired-but-adaptive obstacle avoidance — a small spiking neural network (SNN) using genuine Leaky Integrate-and-Fire (LIF) neuron dynamics, plus a biologically-inspired learning rule (R-STDP) that lets the wiring itself adjust slowly over time based on dopamine. This is the "reflex" layer combined with `total_reflex` in `carl_simulation.py` — it's meant to react faster and more reliably than the reservoir for immediate danger, the same way a spinal reflex arc in a real animal can jerk your hand off a hot stove before your brain has consciously registered pain.

What "spiking" and "LIF" mean, briefly, before the code

Most neural networks (including the reservoir in `carl_reservoir.py`) use continuous-valued "rate" neurons — a neuron's output is just a number like 0.73. A **spiking** neuron instead accumulates incoming charge over time in a "membrane potential" variable, and only produces output (a "spike," a binary 1) at the instant that potential crosses a threshold — after which it resets and starts accumulating again. This is much closer to how real biological neurons actually work. "Leaky Integrate-and-Fire" (LIF) is the simplest standard model of this: the membrane potential leaks (decays) toward zero over time if nothing's exciting it, integrates (adds up) incoming input, and fires when it crosses a threshold.

Lines 1–11: Header and imports

```python
"""
carl_reflex.py — Spiking Neural Network LIF Obstacle Avoidance Layer (END-20

Implements Leaky Integrate-and-Fire (LIF) reflex mechanics.
Uses a differential steering connectivity profile:
  - Obstacles on the left excite the left-turn reflex (slowing the left whee
  - Obstacles on the right excite the right-turn reflex (slowing the right wh
  - If a front collision is imminent (d < 0.35m), triggers a backup-and-spin
"""
import math
import numpy as np
```

The docstring's `0.35m` figure is slightly out of date compared to the actual code below (the real collision threshold, `collision_limit`, is `0.28` normally and `0.20` near food) —

a small documentation drift, not a functional bug, worth knowing if you're asked to cross-reference the docstring against the live thresholds.

`__init__` (lines 19–69)

```python
    def __init__(self, num_sensors=24, num_motor_neurons=2):
        self.num_sensors = num_sensors
        self.num_motors = num_motor_neurons  # [0]: Turn Left, [1]: Turn Righ

        self.V = np.zeros(self.num_motors)
        self.V_th = 1.0
        self.tau_m = 0.02  # 20 ms membrane time constant
        self.dt = 0.033    # 30 Hz control cycle
```

`self.V` is the **membrane potential** for each of the 2 "motor neurons" (one for turn-left, one for turn-right) — starts at zero, no charge accumulated yet. `V_th = 1.0` is the firing threshold. `tau_m = 0.02` (20 milliseconds) is the membrane's own decay time constant — this is deliberately much *faster* than the reservoir's `tau=0.1` (100ms), meaning this reflex layer's internal "memory" of recent input fades roughly 5× quicker — appropriate for something meant to react to *immediate* danger rather than integrate information over a longer timescale.

```python
        self.w = np.zeros((self.num_motors, self.num_sensors))
        angles = np.linspace(-np.pi, np.pi, num_sensors, endpoint=False)

        for j in range(num_sensors):
            a = angles[j]
            if -np.pi/2 <= a <= np.pi/2:
                if a < 0:
                    self.w[0, j] = 3.0 * (1.0 - abs(a) / (np.pi/2))
                elif a > 0:
                    self.w[1, j] = 3.0 * (1.0 - a / (np.pi/2))
                else:
                    self.w[0, j] = 1.8
                    self.w[1, j] = 2.2
```

This hand-designs the *initial* connectivity between sonar rays and the two motor neurons — before any learning happens. Only rays in the front 180° sector (`-π/2` to `+π/2`, i.e. left 90° through right 90°) get any connection at all; rays pointing behind the robot start with zero influence on the reflex. Within that front sector: rays with negative

angle (recall from `carl_simulation.py`'s convention, positive angle = left, so negative = right side of the front arc) excite motor 0 (Turn Left) — meaning **an obstacle detected on the right side should trigger a leftward turn away from it**, which is the correct avoidance direction. The excitation strength scales linearly with how close to dead-center the ray is (`3.0 × (1 - |a|/(π/2))` — a ray pointing straight ahead-right gets nearly full strength 3.0, a ray near the 90° side boundary gets nearly zero) — obstacles more directly ahead matter more than ones barely in the peripheral front zone. The symmetric logic applies for positive angles (left side) exciting motor 1 (Turn Right). The `a == 0` special case (an obstacle detected dead-center, directly ahead) gives *both* motors a nonzero excitation (1.8 and 2.2, asymmetric on purpose) — since turning purely left or right isn't obviously "more correct" for a dead-ahead obstacle, giving both some initial weight lets learning (or the backup logic further down) figure out which response actually works better over time, while a slight asymmetry avoids a perfectly balanced tie.

python

```
        self.trace_pre = np.zeros(self.num_sensors)
        self.trace_post = np.zeros(self.num_motors)
        self.eligibility = np.zeros((self.num_motors, self.num_sensors))

        self.tau_stdp = 0.02
        self.tau_e = 0.1
        self.eta_stdp = 0.01

        self.pre_decay = math.exp(-self.dt / self.tau_stdp)
        self.post_decay = math.exp(-self.dt / self.tau_stdp)
        self.elig_decay = math.exp(-self.dt / self.tau_e)
        self.mem_decay = math.exp(-self.dt / self.tau_m)
```

This block sets up **R-STDP** (Reward-modulated Spike-Timing-Dependent Plasticity) — the learning rule that will slowly adjust `self.w` over time (explained fully at the bottom of `step()`). `trace_pre`/`trace_post` are exponentially-decaying "memory traces" of recent spiking activity — standard STDP implementations track these instead of literally recording every past spike timestamp, since a decaying trace is mathematically equivalent and far cheaper to compute. `eligibility` tracks which specific sensor-to-motor connections were *plausibly responsible* for recent behavior — this is what actually gets modified by dopamine feedback, not the raw traces directly. All four `*_decay` values are precomputed using `math.exp(-dt/tau)` — this is the exact-form solution for exponential decay over a fixed timestep (as opposed to the reservoir's simpler linear-approximation Euler integration in `carl_reservoir.py`) — a more numerically precise way to implement decay when the time constant is known and fixed in advance.

python

```
        self.reflex_outputs = np.zeros(2)
        self.tau_motor = 0.1
        self.motor_decay = math.exp(-self.dt / self.tau_motor)
        self.backup_timer = 0
        self.backup_spin_dir = 1.0
```

`reflex_outputs` is the actual final output of this whole class — the `[left_delta, right_delta]` values `carl_simulation.py` adds into `total_reflex`. It has its *own* separate decay (`tau_motor=0.1`, matching the reservoir's own tau, not the faster membrane tau) — meaning a triggered reflex doesn't snap back to zero instantly, it fades out smoothly over a somewhat longer window than the spike itself, giving a triggered turn enough time to actually have an effect on the robot's heading before disappearing. `backup_timer`/`backup_spin_dir` support the "stuck in a corner" escape behavior further down.

`step()` (lines 71–176) — one tick of the reflex layer

python

```
    def step(self, sensor_dists: np.ndarray, reservoir_bias: np.ndarray, da:
        spikes_pre = np.zeros(self.num_sensors)
        nu_max = 100.0
```

`nu_max = 100.0` — a maximum spike *rate* in Hz, used just below to convert distance into a spiking probability. (Recall from the `carl_simulation.py` walkthrough: `reservoir_bias` is always passed in as `np.zeros(2)` currently — built to allow cognitive influence over this layer, but not yet actually connected to anything nonzero.)

python

```
        d_trigger = 0.45
        collision_limit = 0.28
        if food_dist < 1.0:
            d_trigger = 0.32
            collision_limit = 0.20
```

Two key thresholds, and — importantly — **both get relaxed (made smaller/more permissive) when food is nearby** (`food_dist < 1.0`). This is a deliberate trade-off: normally, CARL should react defensively to anything within 45cm; but if food is within a meter, that threshold drops to 32cm, meaning CARL will tolerate getting closer to things before triggering an avoidance spike — otherwise the obstacle-avoidance reflex could fight against CARL's own attempt to approach and eat nearby food, similar in spirit to the sensory-gating logic seen in `carl_simulation.py`'s food-ray-clearing trick.

```python
        min_front_dist = np.min(sensor_dists[9:15])
        imminent_collision = min_front_dist < collision_limit
```

Checks the narrowest, most-forward six rays (indices 9 through 14, out of 24 evenly spaced rays covering a full circle — this range sits centered on "straight ahead") for the single closest reading. If that's below the collision limit, a frontal collision is imminent.

```python
        min_rear_dist = min(sensor_dists[0], sensor_dists[1], sensor_dists[23
        imminent_rear_collision = min_rear_dist < 0.25
```

Checks specifically the 3 rays nearest to "directly behind" (indices 0, 1, and 23 — since the 24 rays wrap around a circle, index 23 sits right next to index 0 on the other side) for a rear collision threat, using a fixed 0.25m threshold (not adjusted by food proximity — makes sense, since food is presumably ahead/visible, not something you'd be backing into).

```python
        if imminent_collision and self.backup_timer == 0:
            self.backup_timer = 20  # Back up for 20 ticks (~0.6 seconds)
            left_clear = np.mean(sensor_dists[6:12])
            right_clear = np.mean(sensor_dists[12:18])
            self.backup_spin_dir = 1.0 if left_clear > right_clear else -1.0
```

If a frontal collision is imminent *and* there isn't already an active backup sequence in progress (`backup_timer == 0` — this prevents re-triggering and resetting the timer every single tick while stuck against the same wall), start a 20-tick backup maneuver (0.6 seconds at 30Hz — the comment does the arithmetic for you). It decides *which way* to spin/curve during the backup by comparing average clearance on the left side (rays 6–11) versus the right side (rays 12–17) — picking whichever direction has more open space, a sensible heuristic for escaping a dead end.

```python
```

```
        for j in range(self.num_sensors):
            d = sensor_dists[j]
            if d < d_trigger:
                p = nu_max * (1.0 - d / d_trigger) * self.dt
                if np.random.rand() < min(1.0, p):
                    spikes_pre[j] = 1.0
```

This is where sonar readings get converted into actual **spikes** — a genuinely probabilistic, biologically-styled encoding (this style is called a "Poisson spike encoding" in computational neuroscience, though this specific implementation is a simplified version rather than a literal Poisson process). Only rays closer than `d_trigger` produce any spike probability at all. The closer the obstacle, the higher the instantaneous firing probability `p` — at `d=0` (touching), `p = nu_max × 1.0 × dt = 100 × 0.033 ≈ 3.3`, clamped to 1.0 (certain to spike this tick); at `d` approaching `d_trigger`, `p` approaches zero. This means the sonar-to-spike encoding is itself a form of "rate coding" — closer things spike more *often* across many ticks, not with more *intensity* in a single tick (since a spike is just binary 1 or nothing).

python

```
        current_inputs = np.dot(self.w, spikes_pre) + reservoir_bias
        self.V = self.V * self.mem_decay + current_inputs
```

This is the actual LIF integration: `np.dot(self.w, spikes_pre)` computes, for each of the 2 motor neurons, the weighted sum of which sensors just spiked (using the connectivity matrix built in `__init__`, which — remember — also gets adjusted by learning over time). Adding `reservoir_bias` (currently always zero) leaves room for cognitive top-down influence. `self.V * self.mem_decay` is the "leaky" part — the membrane potential decays by the precomputed factor from `tau_m=0.02` every tick, then the new input gets added on top — "leaky integrate."

python

```
        spikes_post = np.zeros(self.num_motors)
        for i in range(self.num_motors):
            if self.V[i] >= self.V_th:
                spikes_post[i] = 1.0
                self.V[i] = 0.0  # reset
```

The "fire" part of leaky-integrate-**and-fire**: if a motor neuron's accumulated potential crosses the threshold (1.0), it spikes and its potential immediately resets to zero (a real neuron can't fire continuously without first "recharging") — this is the standard LIF reset rule.

```python
        self.reflex_outputs *= self.motor_decay

        reflex_strength = 5.0
        clamp_limit = 6.0
        if food_dist < 1.2:
            reflex_strength = 1.0
            clamp_limit = 1.5
```

The output decays smoothly each tick (as described in `__init__`), and — again — the actual strength of any triggered reflex response is drastically reduced when food is close by (1.0 vs. the normal 5.0, a 5× reduction; clamp ceiling reduced from 6.0 to 1.5). This is the same "don't let obstacle-avoidance fight food-seeking" principle applied a second time, at the output-magnitude level rather than just the triggering-threshold level from earlier — a belt-and-suspenders approach to the same design problem.

```python
        if spikes_post[0] > 0:
            self.reflex_outputs[0] -= reflex_strength
            self.reflex_outputs[1] += reflex_strength

        if spikes_post[1] > 0:
            self.reflex_outputs[0] += reflex_strength
            self.reflex_outputs[1] -= reflex_strength
```

If the "Turn Left" motor neuron fired: decrease left wheel speed, increase right wheel speed — this is exactly how a differential-drive robot turns left (imagine the right wheel spinning faster than the left — the robot pivots toward the slower side). The symmetric logic applies for "Turn Right." Both can fire in the same tick (they're independent LIF neurons, not mutually exclusive) — in which case their effects would partially cancel, which is a sensible emergent behavior for a genuinely ambiguous/symmetric threat.

```python
```

```
        if self.backup_timer > 0:
            self.backup_timer -= 1
            if imminent_rear_collision:
                self.reflex_outputs[0] = 3.5 * self.backup_spin_dir
                self.reflex_outputs[1] = -3.5 * self.backup_spin_dir
            else:
                if self.backup_spin_dir > 0:
                    self.reflex_outputs[0] = -4.5
                    self.reflex_outputs[1] = -1.5
                else:
                    self.reflex_outputs[0] = -1.5
                    self.reflex_outputs[1] = -4.5
        elif imminent_rear_collision:
            self.reflex_outputs[0] = 3.0
            self.reflex_outputs[1] = 3.0
```

This is an **emergency override that completely replaces** whatever the spike-driven output computed above (note it's plain assignment (`=`), not addition) — a hard-coded escape behavior layered on top of the learned/spiking system. If a backup sequence is active: check if it's now *also* blocked from behind (`imminent_rear_collision` — meaning CARL is genuinely wedged into a corner, blocked both front and back). If so, spin in place (both wheels driven in opposite directions at equal magnitude, `±3.5`, scaled by whichever direction was chosen as clearer back in the trigger logic) rather than trying to reverse into something. If only blocked in front (the normal case), do an asymmetric reverse-and-swing: both wheels go backward, but one more strongly than the other (`-4.5` vs `-1.5`), which produces a curved backward arc turning toward the clearer side rather than a straight-line reverse — a more effective escape than either pure spin or pure straight backup alone. If there's no backup in progress but a *rear* collision is suddenly imminent (nothing in front, wall approaching from behind), simply drive forward at moderate speed, away from the rear threat.

python

```
        self.reflex_outputs = np.clip(self.reflex_outputs, -clamp_limit, clam
```

Final safety clamp, using whichever `clamp_limit` was set earlier (6.0 normally, 1.5 near food).

R-STDP learning (lines 162–174)

python

```
        self.trace_pre = self.trace_pre * self.pre_decay + spikes_pre
        self.trace_post = self.trace_post * self.post_decay + spikes_post
```

Updates the two decaying spike-history traces — each trace decays a little every tick and gets a `+1` bump wherever a spike just occurred, so these values represent "how recently and strongly did this sensor/motor spike," smoothly blended across time rather than a hard on/off record.

python

```
        for i in range(self.num_motors):
            for j in range(self.num_sensors):
                self.eligibility[i, j] = (self.eligibility[i, j] * self.elig
                                          (self.trace_pre[j] * spikes_post[i]
                                           self.trace_post[i] * spikes_pre[j]
```

This is the actual **STDP** (Spike-Timing-Dependent Plasticity) rule, computed as an eligibility trace rather than applied directly to weights. The term `trace_pre[j] * spikes_post[i]` is positive when sensor `j` recently fired *before* motor `i` fires now — the classic "pre-before-post" pattern that in real STDP theory should *strengthen* that connection (this sensor's activity reliably preceded/predicted this motor's response, so reinforce that link). The term `trace_post[i] * spikes_pre[j]` (subtracted) captures the reverse timing — motor `i` recently fired and *now* sensor `j` fires — the "post-before-pre" pattern that classically *weakens* a connection. This eligibility value doesn't change weights immediately by itself — it's tracking *which* connections are currently "eligible" to be modified, waiting for a reward/dopamine signal to decide whether to actually reinforce or weaken them (hence "eligibility trace," a real, named concept in reinforcement-learning-flavored neuroscience, distinct from plain STDP).

python

```
        da_error = da - 1.0
        self.w += self.eta_stdp * self.eligibility * da_error
        self.w = np.clip(self.w, -3.0, 3.0)

        return self.reflex_outputs.copy()
```

This is the "**R**" in R-STDP — Reward-modulated. `da_error = da - 1.0` compares current dopamine against `1.0`, which is exactly `EndocrineSystem.BASELINE_DA` — meaning this is implicitly a reward-prediction-error-like signal: dopamine above its resting baseline (a recent positive event, like eating) produces a positive `da_error`, multiplying the eligibility trace and *actually adjusting the weights* (`self.w`) in the direction the eligibility trace indicates. Below-baseline dopamine would push weights the *opposite*

direction. This is how `self.w` — the hand-designed initial connectivity from `__init__` — is meant to slowly self-tune over the robot's operating lifetime: connections that were active right before a rewarding outcome get reinforced, connections active right before a non-rewarding/negative outcome get weakened. `eta_stdp=0.01` is a small learning rate, keeping this adjustment gradual. Final clamp keeps weights bounded, and `.copy()` on the return value prevents the caller from accidentally holding a live reference to (and possibly mutating) this object's internal `reflex_outputs` array.

Summary

This file is genuinely one of the more sophisticated pieces of neuroscience-inspired engineering in the repo: real LIF spiking dynamics (not just an approximation), a real probabilistic distance-to-spike encoding, and a real R-STDP learning rule tying dopamine feedback to synaptic weight changes — layered underneath a completely separate, deterministic "backup and escape" emergency override for the specific case of getting physically stuck. If someone asks "does the reflex layer learn, or is it just fixed rules," the honest, complete answer is both: the base connectivity is hand-designed, but it's continuously nudged by dopamine-modulated STDP while running, and the emergency backup logic sits entirely outside that learning system as a hard override.